Lab Sheet 05

Part 01

Step 1: Load the Required Libraries

```r
library(MASS)
```

The MASS package contains datasets like Boston that we'll use in this lab.

Step 2: Load the Data

```r
data("Boston")
```

The Boston dataset includes housing data with variables like crime rate, tax, and median home value (medv).

Simple Linear Regression

Simple Linear Regression is used to understand the relationship between one independent variable (lstat) and the dependent variable (medv).

Code

```r
fit1 = lm(medv ~ lstat, data = Boston)

summary(fit1)
```

Explanation

- lm() creates a linear model.
- medv ~ lstat means predicting medv using lstat.
- summary(fit1) displays key metrics like coefficients and p-values.

Multiple Linear Regression

This extends simple linear regression by using multiple independent variables.

Code Example 1

```
fit2 = lm(medv ~ lstat + black, data = Boston)

summary(fit2)
```

Code Example 2 (Using All Variables)

```
fit3 = lm(medv ~ ., data = Boston)

summary(fit3)
```

Excluding Specific Variables

```
fit4 = lm(medv ~ . - indus, data = Boston)

summary(fit4)
```

Explanation

- medv ~ . uses all variables except the target (medv).
- \- indus removes the indus variable.

Splitting Data for Training and Testing

Code

```
trainID = sample(1:nrow(Boston), round(0.8 * nrow(Boston)))

train = Boston[trainID, ]

test = Boston[-trainID, ]
```

Explanation

- sample() splits the data into 80% training and 20% testing datasets.

Model Training and Predictions

Code

```
fit = lm(medv ~ ., data = train)

summary(fit)


y_pred = predict(fit, test)

y_actual = test$medv
```

Explanation

- predict() estimates values for the test dataset.

Model Evaluation

Calculate errors to assess model performance.

Code

```
MSE = mean((y_actual - y_pred)^2)

RMSE = sqrt(MSE)

MSE

RMSE
```

Explanation

- **MSE** (Mean Squared Error): Average of squared errors.
- **RMSE** (Root Mean Squared Error): Square root of MSE.

Handling Dummy Variables

Dataset

```
library(pROC)

data("aSAH")
```

```
aSAH$outcome = factor(aSAH$outcome)

aSAH$gender = factor(aSAH$gender)

contrasts(aSAH$gender)
```

Model

```
fit = lm(ndka ~ s100b + outcome + gender, data = aSAH)

summary(fit)
```

Explanation

- Convert categorical variables to factors for regression.
- contrasts() shows how R handles dummy variables.

Part 02

Assumption 1: Linearity

The relationship between the independent and dependent variables should be linear.

Code

```
plot(Boston$lstat, Boston$medv, main = "Scatterplot of lstat vs medv",
   xlab = "lstat", ylab = "medv")

abline(lm(medv ~ lstat, data = Boston), col = "red")
```

Explanation

- A scatterplot shows the relationship between the predictor (lstat) and the response (medv).
- The red line is the fitted regression line. The points should follow a linear pattern.

Assumption 2: Independence

The residuals (errors) should be independent.

Code

```
library(lmtest)

dwtest(lm(medv ~ lstat, data = Boston))
```

Explanation

- The Durbin-Watson test checks for autocorrelation in the residuals.
- A test statistic close to 2 indicates no autocorrelation.

Assumption 3: Homoscedasticity

The residuals should have constant variance.

Code

```
fit = lm(medv ~ lstat, data = Boston)

plot(fitted(fit), residuals(fit), main = "Residuals vs Fitted Values",

   xlab = "Fitted Values", ylab = "Residuals")

abline(h = 0, col = "red")
```

Explanation

- A residuals vs. fitted values plot is used to check homoscedasticity.
- The residuals should be randomly scattered around zero.
- Patterns (e.g., funnel shapes) indicate heteroscedasticity.

Assumption 4: Normality of Residuals

The residuals should be normally distributed.

Code

```
qqnorm(residuals(fit), main = "Normal Q-Q Plot")
qqline(residuals(fit), col = "red")
```

Explanation

- A Q-Q plot compares the distribution of residuals to a normal distribution.
- The points should fall along the red reference line.

Additional Test

```
shapiro.test(residuals(fit))
```

- The Shapiro-Wilk test checks for normality. A p-value > 0.05 indicates normality.

Assumption 5: No Multicollinearity

Independent variables should not be highly correlated.

Code

```
library(car)
vif(lm(medv ~ ., data = Boston))
```

Explanation

- Variance Inflation Factor (VIF) checks multicollinearity.

VIF values > 5 indicate significant multicollinearity